The Midnight Queue: Ranking Suspicious Transactions at Scale

Design and Analysis of Algorithms, Assignment 2. Roll number (digits only): **2013**. Seed s = 2013. Shift capacity K = 25 + (2013 mod 226) = 25 + 205 = **230**. Code: C++17 (repository: github.com/AbdulAzeemHashmi/fraud-risk-ranking-engine). All numbers below come from *results.csv* and the saved experiment files in that repository.

1. Setup, definitions and honest limits

Hardware and software. Dell Latitude 7400, Windows 11 x64, Intel64 Family 6 Model 142 CPU with 8 logical processors (installed RAM could not be read, so it is not reported). MinGW GCC 16.2.0, C++17, *-O2 -Wall -Wextra -Wpedantic*. Timing uses *std::chrono::steady_clock*, excludes file reading and risk score computation, uses a fresh copy of identical input for every contender, and reports the median of 5 runs. **Counting rule (one definition for all contenders).** A comparison is one call of the ranking comparator. A move is one assignment of a record into a slot owned by the algorithm; a swap counts as 3 moves. Memory values are estimates of algorithm storage (bytes), not operating system readings. **Clash with the brief, and my choice.** The brief sets the DNF limit at 5 minutes. I used **30 seconds** per run so the whole tournament could finish; a run above 30 s is marked DNF, and a DNF is never used as a median. Because of this, some points that the brief would have timed are DNF here. **Correctness.** Unit tests compare all five sorts and both Top K methods with a trusted reference (std::sort, used only to check) on empty input, one record, all-equal keys, duplicate-heavy input and band-only keys. Every experiment run also checks that each sort output is ordered, and every Top K output is compared with a Merge Sort reference, which stops the program on any difference. Because the triage key is a total order, an ordered permutation is identical to the reference. Full output comparison with the reference on every condition and size is not a separate test; the ordered check in the runner covers it.

2. Problem A: Know Your Data

The file has 284,807 transactions and 492 frauds (0.173%). There are no missing, non-numeric or non-finite cells, so no row was dropped or filled. There are 1,081 repeated full rows; I kept them because each row is a separate transaction with its own txn_id. Standard deviation uses the population formula (ddof = 0). The risk band table (frauds are used here for evaluation only):

Risk band	0	1	2	3	4	5	6	7	8	9
Transactions	0	0	21,156	107,410	113,116	32,084	6,006	2,797	1,376	862
Share (%)	0	0	7.43	37.71	39.72	11.27	2.11	0.98	0.48	0.30
Frauds	0	0	0	13	30	23	56	97	112	161
Fraud rate in band (%)	0	0	0	0.012	0.027	0.072	0.93	3.47	8.14	18.68

Bands 6 to 9 hold 426 of the 492 frauds (86.6%) in only 11,041 transactions (3.88%). There are **32,767** distinct Amount values. The 10 most frequent values (1.00, 1.98, 0.89, 9.99, 15.00, 0.76, 10.00, 1.29, 1.79, 0.99) cover 46,398 rows, which is **16.29%** of the file (the value 1.00 alone covers 13,688 rows). So Amount has many ties, and the Time and txn_id tie breakers matter. **Section 3.4 properties of the raw file.** The raw file already has C1: Time is nondecreasing (checked on all rows). It does not have C3: only 1 of 284,807 records is in its triage position. It does not have C4: it is not the exact reverse of the triage order. C2, C5 and C6 are built from the same records by my code; they are not properties of the raw file.

3. Problem B: The Contenders

Implemented by me in *algorithms.hpp*: Insertion Sort; Merge Sort (top-down, one scratch array); Naive Quick Sort (last element pivot, Lomuto partition); Hardened Quick Sort (median-of-three pivot with three-way partition); Heap Sort (max-heap, sift-down); Top K with a size-K min-heap; Top K with a bounded insertion buffer of size K. No library sort is inside any contender. **Recursion depth.** Both Quick Sorts recurse into the smaller side and loop on the larger side, so the stack stays O(log n) and nothing crashes on sorted input. The Naive version is still quadratic in time there, so it ends as DNF, not as a crash.

4. Problem C: The Full Ranking Tournament

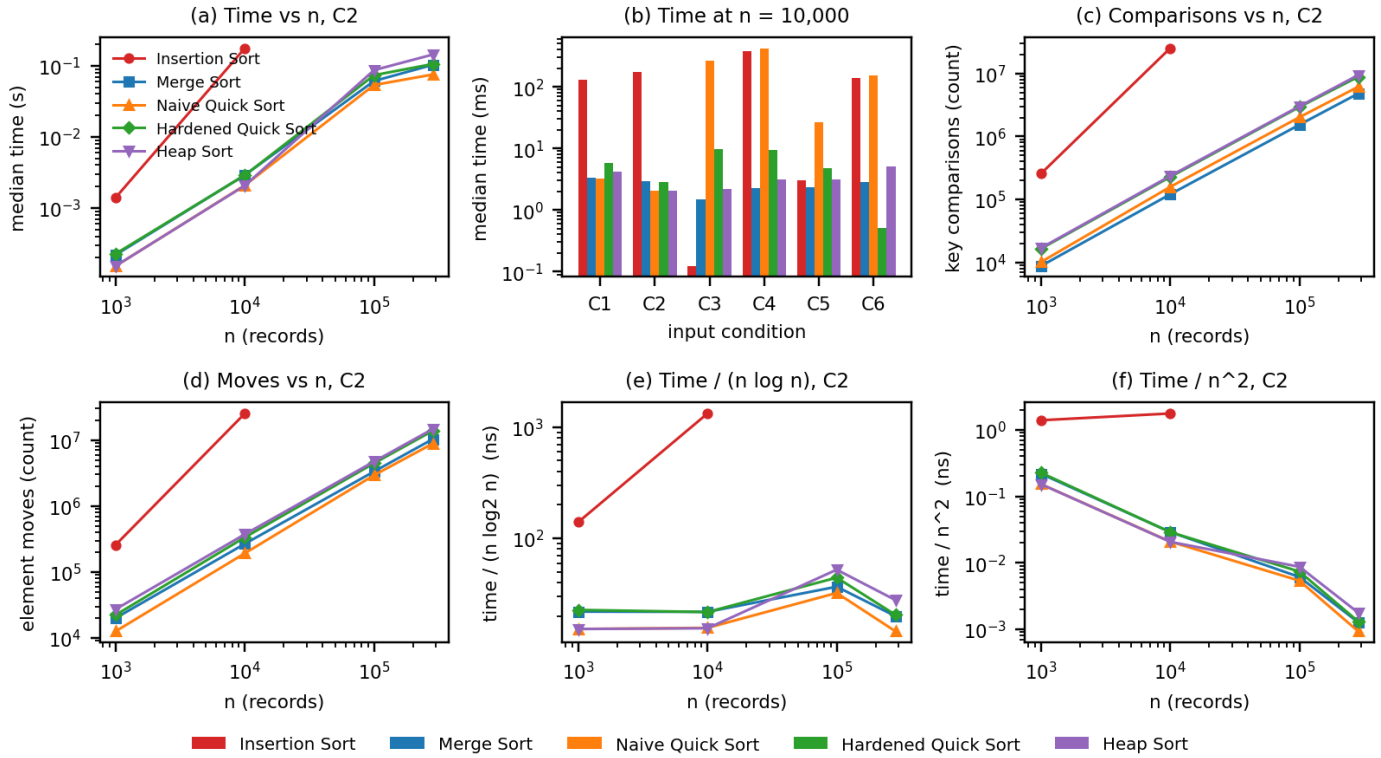


Figure 1. *Problem C plots (median of 5 runs, log scales where shown). (a) time vs n on C2; (b) time at n = 10,000 across C1 to C6; (c) comparisons and (d) moves vs n on C2; (e) time / (n $\log_2$ n) and (f) time / n^2 vs n on C2. Insertion Sort on C2 has no point above n = 10,000 because the larger runs are DNF (above 30 s).*

Contender (n = 284,807)	C1	C2	C3	C4	C5	C6	Extra mem (B)	Stable
Insertion Sort	DNF (over 30 s)	DNF (over 30 s)	0.0042 s 285k cmp 570k mv	DNF (over 30 s)	2.9467 s 262.36M cmp 262.64M mv	DNF (over 30 s)	0	yes
Merge Sort	0.0961 s 4.79M cmp 10.34M mv	0.1030 s 4.80M cmp 10.34M mv	0.0541 s 2.54M cmp 10.34M mv	0.0541 s 2.63M cmp 10.34M mv	0.1339 s 4.01M cmp 10.34M mv	0.0778 s 4.55M cmp 10.34M mv	9,113,824	yes
Naive Quick Sort	0.0781 s 6.60M cmp 9.21M mv	0.0752 s 6.16M cmp 8.86M mv	DNF (over 30 s)	DNF (over 30 s)	1.5175 s 157.02M cmp 3.06M mv	DNF (over 30 s)	608	no
Hardened Quick Sort	0.1380 s 8.90M cmp 13.49M mv	0.1058 s 8.88M cmp 13.83M mv	1.1412 s 162.66M cmp 242.05M mv	1.5574 s 116.33M cmp 172.34M mv	0.2217 s 16.32M cmp 23.66M mv	0.0239 s 1.01M cmp 929k mv	608	no
Heap Sort	0.1365 s 9.44M cmp 14.70M mv	0.1437 s 9.45M cmp 14.73M mv	0.0703 s 9.70M cmp 15.34M mv	0.1389 s 9.21M cmp 14.11M mv	0.1566 s 9.56M cmp 15.11M mv	0.1034 s 7.39M cmp 11.18M mv	0	no

Table 1. *Time, comparisons (cmp), moves (mv), extra memory and stability at the full size. The same table for n = 1,000, 10,000 and 100,000 (all 120 condition and size rows) is in C_summary.csv and results.csv. Memory is an estimate and is the same for every condition.*

What the data says. On shuffled data at n = 284,807, Naive Quick Sort was fastest (0.075 s), then Merge Sort (0.103 s), Hardened Quick Sort (0.106 s) and Heap Sort (0.144 s); Insertion Sort was DNF. Merge Sort never failed and stayed between 0.054 s and 0.134 s on all six conditions at the full size, and Heap Sort between 0.070 s and 0.157 s. Naive Quick Sort was DNF on C3, C4 and C6 at the full size (C3 and C4 at n = 100,000 too). Insertion Sort was best on pre-ranked C3 (0.0042 s, exactly n - 1 = 284,806 comparisons) but needed 2.95 s on nearly ranked C5 against 0.134 s for Merge Sort. Hardened Quick Sort was the best on band-only C6 (0.024 s, 1.01M comparisons).

Clash 1: Hardened Quick Sort on C3 and C4. I expected median-of-three with three-way partition to stay near n log n on ranked input. It did not: on C3 it needed 1.13M, 34.3M and 162.7M comparisons at n = 10,000, 100,000 and 284,807 (1.14 s at the full size, 21 times slower than Merge Sort at 0.054 s), growth close to $n^{1.5}$. On C4 it took 1.56 s. I did not prove the cause. A likely reason is that the three-way partition swaps records from the far end, which scrambles the upper part into a pattern that median-of-three picks badly. So Hardened Quick Sort is hardened against ties (C6) but not against ranked input in my implementation. **Clash 2: n = 100,000 point.** Several contenders have a higher time / (n log n) at n = 100,000 than at n = 284,807 (Figure 1e). I did not find the cause (cache or CPU speed changes on a laptop are possible); I report it as measured. **Predictions for DNF points.** Insertion Sort on C2 is quadratic: n(n-1)/4 inversions, about 2.5 billion at n = 100,000 and 2.0 x 10^{10} at 284,807. At the measured speed of 10.7 ns per step (C6, n = 100,000) that is about 27 s, which matches a run just above my 30 s limit, and about 220 s at 284,807. Naive Quick Sort on C3 needs n(n-1)/2 comparisons (exactly 49,995,000 at n = 10,000 in my data); at 5.4 ns each that is about 27 s at 100,000 and about 219 s at 284,807, again above 30 s. For C6 see M6.

5. Problem D: The Shift Queue

K	n=100,000 heap	buffer	full sort	n=284,807 heap	buffer	full sort
10	0.0010	0.0074	0.0554	0.0026	0.0220	0.1754
50	0.0010	0.0376	0.0553	0.0027	0.1055	0.1750
230 (your K)	0.0012	0.1616	0.0560	0.0030	0.4830	0.1740
500	0.0017	0.3551	0.0552	0.0035	1.0564	0.1763
2,068 (half frauds)	-	-	-	0.0067	5.0838	0.1872
5,000	0.0079	3.4552	0.0556	0.0138	DNF	0.1782
10,000 (n/10)	0.0136	6.8542	0.0562	-	-	-
28,480 (n/10)	-	-	-	0.0438	DNF	0.1954

Table 2. *Median time in seconds on C2 (shuffled). A dash means that K was not part of that size. C1 gives the same order of methods (for example K = 2,068, n = 284,807: heap 0.0041 s, full sort 0.1017 s, buffer 5.10 s); see D_summary.csv.*

Where does each method stop being the best? The size-K min-heap was the fastest method at every tested point, even at K = n/10, so I found **no crossover away from the heap**. What happens when K is close to n was not measured. The bounded insertion buffer beats the full sort only for small K: at n = 100,000 it wins at K = 50 (0.0376 s against 0.0553 s) and loses at K = 230 (0.162 s against 0.056 s); at n = 284,807 the same switch happens between K = 50 and K = 230. It reached the 30 s limit at n = 284,807 for K = 5,000 and K = 28,480. **Fraud capture and cost of half capture (Figure 2b).** Using Class only to evaluate, the smallest K that captures at least half of the 492 frauds is **K = 2,068** (246 frauds). The cost to build that queue at n = 284,807 (C1 / C2): full sort 0.102 s / 0.187 s, 4.79M / 4.80M comparisons, 9.1 MB extra; size-K heap 0.0041 s / 0.0067 s, 527k / 519k comparisons, 66 KB; bounded buffer 5.10 s / 5.08 s, 575M comparisons, 66 KB. **Clash 3: ranking rule against fraud capture.** My own K = 230 queue captures only 3 frauds (0.61%); K = 500 captures 63 (12.8%), K = 5,000 captures 361 (73.4%) and K = 28,480 captures 438 (89.0%). The fastest algorithm cannot fix this, because the weakness is the scoring rule. Inside band 9 (862 records) the order is by Amount, so large normal payments probably come first (I did not test this reason). I keep the rule as given, because the assignment fixes the sort key, and I report this limit to the platform lead in Problem H.

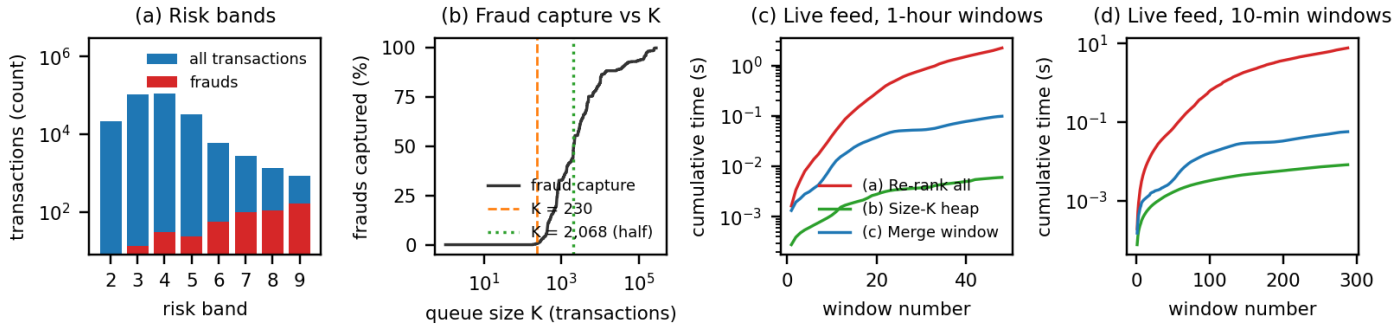


Figure 2. *(a) Transactions and frauds per risk band (count, log scale). (b) Percent of all 492 frauds captured in the top K (K in transactions, log scale); dashed line is K = 230, dotted line is K = 2,068. (c) and (d) Problem E: cumulative time in seconds (log scale) against window number for 1 hour and 10 minute windows; red is re-rank all, green is the size-K heap, blue is merge window into top K.*

6. Problem E: The Live Feed

Strategy	1-hour windows (48): final time (s)	peak extra memory (B)	10-minute windows (288): final time (s)	peak extra memory (B)
(a) Re-rank all with Merge Sort	2.2211	35,006,336	7.7774	20,364,160
(b) Maintain size-K heap	0.0060	14,720	0.0082	14,720
(c) Merge Sort window, merge into top K	0.0983	1,055,936	0.0568	269,504

Table 3. *Live feed over 48 hours, median of 5 complete runs (no time limit in this problem).*

Strategy (b) was fastest and used least memory for both window sizes: 0.0060 s against 2.22 s for re-ranking everything (370 times faster) with 1-hour windows, and 0.0082 s against 7.78 s (950 times) with 10-minute windows. Strategy (c) is far better than (a) (0.098 s and 0.057 s) but slower than (b). Shorter windows hurt (a) most: its cost grew 3.5 times when windows became 6 times more frequent. Note: the saved peak memory for (a) differs between the two window sizes (35.0 MB and 20.4 MB) although both end with the full history; I did not investigate this.

7. Problem F: The Mathematical Case

M1 Insertion Sort. The inner loop shifts one record for each inversion, so the cost is Θ(n + I), where I is the number of inversions. Best case (ranked): I = 0, so n - 1 comparisons, Θ(n) (measured 284,806 on C3). Worst case (reverse): I = n(n-1)/2, $\Theta(n^2)$. Average (random order): each pair is inverted with probability 1/2, so I = n(n-1)/4, $\Theta(n^2)$ (C2, n = 10,000: predicted 25.0M comparisons, measured 25.3M). **Inversions on my C5 sample.** My code counts one move to lift each record, one to place it, and one per shift, so moves = 2(n-1) + I. This gives I = 2,894 (n = 1,000), 322,589 (10,000), 34,880,909 (100,000) and **262,073,628** (284,807). Check: comparisons = I + n - 1 = 262,358,434, equal to the measured count. Model: 1% of the records jump to random places, each crossing about n/3 records, so I is about $n^2/300 = 2.70 \times 10^8$ at 284,807, within 3% of the measured value.

M2 Merge Sort. $T(n) = 2T(n/2) + cn$. Iteration: $T(n) = 2^k T(n/2^k) + kcn$. With $k = \log_2 n$ this is $nT(1) + cn \log_2 n = \Theta(n \log n)$. Master theorem: $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, so case 2 gives Θ(n log n). Measured on C2 at 284,807: 4.80M comparisons, against $n \log_2 n - n = 4.88$M.

M3 Quick Sort. (a) Worst case: $T(n) = T(n-1) + cn = T(n-2) + c(n-1) + cn = \ldots = c(n + (n-1) + \ldots + 1) = cn(n+1)/2 = \Theta(n^2)$. Measured on C3: exactly n(n-1)/2 comparisons (499,500 at n = 1,000 and 49,995,000 at 10,000). (b) Best case: $T(n) = 2T(n/2) + cn$. The recursion tree has level j with 2^j nodes of size $n/2^j$, so each of the $\log_2 n + 1$ levels costs cn, total Θ(n log n). (c) 1:9 split: $T(n) = T(n/10) + T(9n/10) + n$. Every level costs at most n. The shallowest branch ends at depth $\log_{10} n$ and all levels above it are full, so the cost is at least $n \log_{10} n$. The deepest branch ends at depth $\log_{10/9} n = \ln n / \ln(10/9)$, about 9.49 ln n, so the cost is at most $n \log_{10/9} n$. Both bounds are n log n times a constant, so T(n) = **Θ(n log n)**.

M4 Heap Sort. Building the heap: a node of height h costs at most 2h comparisons in sift-down, and at most $n/2^{h+1}$ nodes have height h, so the cost is the sum over h of $(n/2^{h+1})\ 2h = n$ times the sum of $h/2^h$, which is at most 2n, so Θ(n). The extraction phase does n - 1 sift-downs of at most $2 \log_2 n$ comparisons each, so Θ(n log n); total Θ(n log n). Measured at 284,807 on C2: 9.45M comparisons, $1.83\ n \log_2 n$, below the bound $2\ n \log_2 n$.

M5 Top K. (a) Full sort then take K: Θ(n log n + K), almost independent of K (Table 2 shows 0.055 s to 0.056 s at n = 100,000). (b) Size-K heap: every record costs one comparison with the root, and a record that beats the root costs O(log K). Worst case Θ(n log K). In random order only about K ln(n/K) records replace the root, so the expected cost is Θ(n + K log K log(n/K)), plus Θ(K log K) to sort the heap at the end. (c) Bounded insertion buffer: each record scans the sorted list of K records, so worst case Θ(nK); my scan starts at the top, so most records scan all K entries (n = 100,000, K = 230: 22.8M comparisons against nK = 23.0M). **Crossovers for n = 100,000.** Buffer against full sort, by comparison counts: $nK = n \log_2 n - n$ gives K about 15 (the full sort used 1,536,322 comparisons). Measured by time: between K = 50 and K = 230. **Clash 4:** the counts predict a crossover too early, because Merge Sort makes about 2 moves and a scratch copy for every comparison, while the buffer scan is a cheap sequential loop (7.5 ns against 36 ns per comparison at K = 50). So I trust the measured crossover. Heap against full sort: the heap needs fewer comparisons than the full sort even at K = n/10 (889,573 against 1,536,322), so I predict a crossover only for K much larger than n/10; I measured none. Buffer against heap: predicted crossover at K = 1 or 2; at K = 10 the heap was already faster (0.00096 s against 0.00736 s).

M6 Naive Quick Sort on C6. Let the groups, in priority order, have sizes $m_1 \ldots m_d$. The comparator says "a before b" only when the band is strictly higher, so with Lomuto partition all records equal to the pivot go to the right side, and the pivot sits at the border. (1) Two records of the same group stay together until one of them is a pivot, so each such pair is compared exactly once: sum of $m_i(m_i - 1)/2$ comparisons. (2) Take u in group a and v in a lower group b. They are separated (never compared) if a pivot comes from a group strictly between, or from group b other than v (u goes left, v goes right). They are compared if u or v is the first pivot among these candidates, which has probability $2/(m_b + 1 + B_{ab})$, where B_{ab} is the total size of groups between a and b, assuming pivots arrive in random order. So $\mathbf{E[comparisons] = \sum_i m_i(m_i - 1)/2 + \sum_{a<b} m_a m_b \cdot 2/(m_b + 1 + B_{ab})}$. The first term dominates: it is about $0.319\ n^2/2$, because the sum of squared band shares is 0.319. I used the group sizes n times the full-file band shares of Problem A (the exact group sizes of each sample were not saved), and compared with the measured comparisons:

n	Predicted comparisons	Measured comparisons	Difference
1,000	161,111	162,599	+0.9%
10,000	15,955,135	15,959,396	+0.03%
100,000	1,593,938,359	1,592,991,850	-0.06%
284,807	12,928,313,287	DNF (above 30 s)	about 133 s at 10.3 ns per comparison

Table 4. *M6 prediction against measurement for Naive Quick Sort on C6.*

M7 Live feed. Let n records arrive in w windows of n/w records. (a) Re-rank all: after window j sort jn/w records, total about the sum of (jn/w) log n = Θ(w n log n). (b) Size-K heap: each record is offered once, Θ(n log K) worst case, plus Θ(K log K) to publish the sorted queue after each window, total Θ(n log K + wK log K). (c) Sort each window, merge into the top K list: w (n/w) log(n/w) + w O(K + n/w) = Θ(n log(n/w) + wK). Strategy (a) is never best for w of 2 or more (extra factor about w/2). Strategy (b) beats (c) while w K ($\log_2$ K - 1) is less than n ($\log_2$(n/w) - 1); for n = 284,807 and K = 230 this holds until w is about 1,200 (windows of about 2.4 minutes), after which (c) becomes better. Both tested sizes (w = 48 and 288) lie before this point, and (b) won, as predicted. **Clash 5:** the worst case formulas give about 2.8M for (b) and 2.9M for (c) comparisons at w = 288, yet (b) measured 7 times faster. The formulas assume every record causes a heap update; in practice most records are rejected with one comparison, and Merge Sort copies records, so I trust the measurement.

M8 Space. Insertion Sort Θ(1) (0 B measured). Merge Sort Θ(n) (9,113,824 B = 32 B x 284,807). Naive and Hardened Quick Sort Θ(log n) stack (608 B), because I recurse on the smaller side (without that trick the Naive stack could reach Θ(n)). Heap Sort Θ(1) (0 B). Top K: full sort Θ(n); size-K heap Θ(K) (66,176 B = 32 B x 2,068); insertion buffer Θ(K). Live feed: (a) Θ(n), (b) Θ(K), (c) Θ(K + n/w).

8. Problem G: Change Request

The change request was not posted when this report was generated, so this addendum is empty. It will say what changed, which conclusions in Sections 4 to 6 still hold, and give fresh measurements from the same C++ runner.

9. Problem H: Recommendation for the Platform Lead

1. Overnight batch (a full day, about 285,000 records): ship Merge Sort. Evidence: it finished every condition at the full size in 0.054 s to 0.134 s, with Θ(n log n) in the best and worst case (M2). Naive Quick Sort was fastest on shuffled input (0.075 s) but did not finish on three of six conditions, and my Hardened Quick Sort took 1.1 to 1.6 s on ranked input. Heap Sort is a safe second choice (0.070 s to 0.157 s, no extra memory). Assumption: 9.1 MB of scratch memory is acceptable and the day has about this many records. **Wrong if** memory is very tight (then use Heap Sort), or if the input is almost always already ranked (then Insertion Sort took 0.0042 s on C3, but 2.95 s on C5, so check first).

2. Shift queue of K = 230: ship the size-K min-heap. Evidence: 0.0030 s against 0.174 s for a full sort at n = 284,807 (57 times faster) and 66 KB against 9.1 MB of memory; it was fastest at every tested K, up to n/10 (Table 2). The bounded insertion buffer took 0.48 s at K = 230 and 5.1 s at K = 2,068, and is not recommended. Assumption: K is much smaller than n and the queue is built once from a full batch. **Wrong if** K grows toward n (I did not measure K above n/10), or if the analysts need the full ranked list anyway; then a full Merge Sort is simpler. Important warning for the business: with the fixed triage rule, the K = 230 queue contains only 3 of 492 frauds (0.61%); half of the frauds need K = 2,068

(Figure 2b). Choosing a faster algorithm will not change this; changing the scoring rule would.

3. Live refresh every window: ship the size-K heap fed by new records only (strategy b). Evidence: over 48 hours it took 0.0060 s (1-hour windows) and 0.0082 s (10-minute windows), against 2.22 s and 7.78 s for re-ranking everything, with 14,720 B of memory. Strategy (c) took 0.098 s and 0.057 s. Assumption: a record never leaves or changes after it arrives, and w stays below about 1,200 windows for this n and K (M7). **Wrong if** old records must expire (for example a rolling 24-hour queue) or be corrected, because a heap cannot cheaply remove them; then keep sorted windows and merge them (strategy c) or rebuild from the stored windows.

10. Bonus: Analyst Console, and AI use disclosure

Analyst Console. A local web page (served by a small Python script; all ranking is done by a C++ program) lets an analyst choose Full Sort with Merge Sort, size-K min-heap or bounded insertion buffer, set K (default 230), and see the top K transactions with transaction ID, risk band, Amount and Time, plus the run time, comparison count and move count of that run. Bonus marks depend on Problems A to H being complete (Problem G is pending) and on the viva demonstration. **AI use disclosure.** I used **DeepSeek** to understand the problem statement, **Codex** for vibe coding (the C++ code, tests, experiment runner, the Python profile and the console), and **Claude** for file generation (this report and its figures). The measurements come from running the code on my own computer; the data file is not included in the submission, and no machine learning model, extra dataset or sorting library was used inside the contenders.

Appendix: Prediction Log (Problem C)

#	Cond	n	Predicted winner / loser	Actual winner / loser	Win ok	Lose ok
1	C1	1,000	Merge / Insertion	Heap / Insertion	no	yes
2	C2	1,000	Hardened Quick / Insertion	Heap / Insertion	no	yes
3	C3	1,000	Insertion / Naive Quick	Insertion / Naive Quick	yes	yes
4	C4	1,000	Heap / Naive Quick	Merge / Insertion	no	no
5	C5	1,000	Insertion / Naive Quick	Insertion / Naive Quick	yes	yes
6	C6	1,000	Hardened Quick / Naive Quick	Hardened Quick / Insertion	yes	no
7	C1	10,000	Merge / Insertion	Naive Quick / Insertion	no	yes
8	C2	10,000	Hardened Quick / Insertion	Heap / Insertion	no	yes
9	C3	10,000	Insertion / Naive Quick	Insertion / Naive Quick	yes	yes
10	C4	10,000	Heap / Naive Quick	Merge / Naive Quick	no	yes
11	C5	10,000	Insertion / Naive Quick	Merge / Naive Quick	no	yes
12	C6	10,000	Hardened Quick / Naive Quick	Hardened Quick / Naive Quick	yes	yes

#	Cond	n	Predicted winner / loser	Actual winner / loser	Win ok	Lose ok
13	C1	100,000	Merge / Insertion	Hardened Quick / Insertion	no	yes
14	C2	100,000	Hardened Quick / Insertion	Naive Quick / Insertion	no	yes
15	C3	100,000	Insertion / Naive Quick	Insertion / Naive Quick	yes	yes
16	C4	100,000	Heap / Naive Quick	Merge / Naive Quick	no	yes
17	C5	100,000	Insertion / Naive Quick	Merge / Naive Quick	no	yes
18	C6	100,000	Hardened Quick / Naive Quick	Hardened Quick / Insertion	yes	no
19	C1	284,807	Merge / Insertion	Naive Quick / Insertion	no	yes
20	C2	284,807	Hardened Quick / Insertion	Naive Quick / Insertion	no	yes
21	C3	284,807	Insertion / Naive Quick	Insertion / Naive Quick	yes	yes
22	C4	284,807	Heap / Naive Quick	Merge / Naive Quick	no	yes
23	C5	284,807	Insertion / Naive Quick	Merge / Insertion	no	no
24	C6	284,807	Hardened Quick / Naive Quick	Hardened Quick / Naive Quick	yes	yes

Reasons written before each round (same for each condition at all four sizes). C1: arrival order has no known link to the triage key, so Merge Sort keeps n log n. C2: shuffle removes order; three-way partition protects Quick Sort; insertion shifts many records. C3: ranked input needs few insertion shifts; the last pivot on sorted data is unbalanced. C4: Heap Sort keeps n log n; reverse order is the worst pattern for the last pivot. C5: only 1% moved, so insertion shifts stay limited; the last pivot is sensitive to ordered runs. C6: many ties; three-way partition handles equal keys while Lomuto repeatedly splits off one record.

What I got wrong. The winner was right in 9 of 24 rounds and the loser in 20 of 24. (1) I picked Merge Sort or Hardened Quick Sort for C1 and C2, but Heap Sort, Naive Quick Sort or Hardened Quick Sort won depending on size; on random data Naive Quick Sort is cheap per step and does not suffer from ties, so it often won. (2) I picked Insertion Sort for C5; it won only at n = 1,000. At larger n the 1% displaced records create I = $n^2/300$ inversions (262 million at 284,807), so Merge Sort won. (3) I picked Heap Sort for C4; Merge Sort won every time. (4) I picked Naive Quick Sort as loser for C4 and C6 at small sizes, but Insertion Sort lost at n = 1,000 on C4 and C6 and at n = 100,000 on C6 (it does more comparisons than Naive Quick Sort there). (5) I did not predict that my Hardened Quick Sort would be slow on C3 and C4 (Clash 1).